

OOP and Modules

Course: Rust Basics

Author: Aleksandr Kugushev

Object-Oriented Programming

- Encapsulation
- Inheritance
- Polymorphism

OOP => Object-Oriented Design

- Cashier
- Merchandiser
- Supervisor
- Laborer
- Director

OOP => Object-Oriented Design

- Employees

- Cashier
- Merchandiser
- Supervisor
- Laborer
- Director



- Customer

- Fire

- Salary

- ...

- Buy Goods

- Provide Feedback

- ...

Encapsulation

```
struct MerchandiserImpl {  
    name: String,  
    age: u8,  
}  
  
impl MerchandiserImpl {  
    fn get_name(&self) -> &String { &self.name }  
    fn get_age(&self) -> u8 { self.age }  
}
```

Encapsulation

```
struct MerchandiserImpl {  
    name: String,  
    age: u8,  
}  
  
impl MerchandiserImpl {  
    fn is_full_age(&self) -> bool { self.age >= 18 }  
}
```

Inheritance

```
trait Employee {  
    fn fire(self);  
    fn pay_salary(&mut self);  
    fn get_age(&self) -> u8;  
}  
  
trait Merchandiser: Employee {  
    fn stock_items(&self, items: Goods);  
}
```

Multiple Inheritance

```
trait Customer {  
    fn buy_goods(&mut self, goods: Goods);  
}
```

```
trait Merchandiser: Employee + Customer {  
    fn stock_items(&self, items: Goods);  
}
```

Polymorphism (static)

```
trait Merchandiser: Employee + Customer
```

```
impl Employee for MerchandiserImpl {  
    fn fire(self) { todo!() }  
    fn pay_salary(&mut self) { todo!() }  
    fn get_age(&self) -> u8 { todo!() }  
}
```

```
impl Customer for MerchandiserImpl {  
    fn buy_goods(&mut self, goods: Goods) { todo!() }  
}
```

```
impl Merchandiser for MerchandiserImpl {  
    fn stock_items(&self, items: Goods) { todo!() }  
}
```

Polymorphism (static)

```
impl Employee for MerchandiserImpl {  
    fn fire(self) { println!("Ok :`(") }
```

...

```
impl Employee for DirectorImpl {  
    fn fire(self) { println!("No, YOU'RE FIRED!") }
```

...

Impl

```
fn do_fire(employee: impl Employee) {  
    employee.fire()  
}  
  
fn is_full_age(employee: &impl Employee) -> bool {  
    employee.get_age() > 18  
}  
  
fn make_happy(employee: &mut impl Employee) {  
    employee.pay_salary()  
}  
  
fn check_discount(employee: &impl Employee + Customer)
```

```
impl MyTrait = <T: MyTrait>
```

```
fn do_fire(employee: impl Employee) {  
    employee.fire()  
}
```

```
fn do_fire<T: Employee>(employee: T) {  
    employee.fire()  
}
```

Blanket Implementation

```
impl From<&str> for MerchandiserImpl {  
    fn from(value: &str) -> Self {  
        Self {  
            name: value.to_string(),  
            age: 42,  
        }  
    }  
}
```

```
let merchandiser: MerchandiserImpl = "Ivan".into();
```

Blanket Implementation

```
impl<T, U> Into<U> for T
  where U: From<T>,
{
  fn into(self) -> U {
    U::from(self)
  }
}
```

Practice: Blanket Implementation

```
trait Hp {  
    fn suffer(&mut self, damage: u32);  
}
```

```
trait Damage {  
    fn damage(&self) -> u32;  
}
```

```
trait Attack {  
    fn attack<T: Hp>(&self, victim: &mut T);  
}
```

```
struct Hero;  
struct Dummy;
```

```
let hero = Hero;  
let mut dummy = Dummy;  
hero.attack(&mut dummy);
```

Practice: Blanket Implementation

```
impl Hp for Dummy {  
    fn suffer(&mut self, damage: u32) {}  
}  
  
impl Damage for Hero {  
    fn damage(&self) -> u32 { 42 }  
}  
  
impl<V> Attack for V  
where  
    V: Damage,  
{  
    fn attack<T: Hp>(&self, victim: &mut T) {  
        victim.suffer(1);  
    }  
}
```

Polymorphism (dynamic)

```
let employees: Vec<Employee> = vec![  
    MerchandiserImpl { name: String::from("Ivan"), age: 42 },  
    DirectorImpl { full_name: String::from("Aleksandr  
Aleksandrovich") },  
];
```

error[E0277]: the size for values of type `dyn Employee` cannot be known at compilation time

Sized

```
struct MerchandiserImpl {  
    name: String,  
    age: u8,  
}
```

```
struct DirectorImpl {  
    full_name: String,  
}
```

Dynamically Sized Types

```
let str: &str = "Hello World!";
```

- Pointer to data
- Size

The golden rule of dynamically sized types is that we must always put values of dynamically sized types behind a pointer of some kind.

Sized trait

```
fn generic<T>(t: T) {  
}
```

```
fn generic<T: Sized>(t: T) {  
}
```

```
fn generic<T: ?Sized>(t: &T) {  
}
```

```
fn generic<T: ?Merchandiser>(t: &T) {  
}
```

?Sized in Vec and Box

```
pub struct Vec<T>{  
    ...  
}
```

```
pub struct Box<T: ?Sized> {  
    ...  
}
```

?Sized Employees

```
let employees: Vec<Employee> = vec![  
  MerchandiserImpl { name: String::from("Ivan"), age: 42 },  
  DirectorImpl { full_name: String::from("Aleksandr Aleksandrovich") },  
];
```

error[E0782]: trait objects must include the `dyn` keyword

```
let employees: Vec<Box<dyn Employee>> = vec![  
  Box::new(MerchandiserImpl { name: String::from("Ivan"), age: 42 }),  
  Box::new(DirectorImpl { full_name: String::from("Aleksandr  
Aleksandrovich") }),  
];
```

dyn keyword

- Prefix of a trait object's type
- Two pointers
 - Pointer to data
 - Pointer to vtable

```
size_of::<Box<dyn ...>> ( )  
128
```

dyn Employees

```
let employees: Vec<Box<dyn Employee>> = vec![
    Box::new(MerchandiseImpl { name: String::from("Ivan"), age:
42 }),
    Box::new(DirectorImpl { full_name: String::from("Aleksandr
Aleksandrovich") }),
];

trait Employee {
    fn praise(&self) { println!("Thank you") }
}

impl Employee for DirectorImpl {
    fn praise(&self) {
        println!("Give me money, no praise!");
    }
}
```

dyn Employees

```
let employees: Vec<Box<dyn Employee>> = vec![  
    Box::new(MerchandiseImpl { name: String::from("Ivan"), age:  
42  } ),  
    Box::new(DirectorImpl { full_name: String::from("Aleksandr  
Aleksandrovich")  } ),  
];
```

```
for employee in employees {  
    employee.praise();  
}
```

> Thank you

> Give me money, no praise!

dyn

```
fn dyn_fun(employee: &dyn Employee) {  
    employee.praise();  
}
```

```
dyn_fun(employees[0].borrow());
```

```
let employee: &dyn Employee = employees[0].borrow();  
employee.praise();
```

Static vs Dynamic Polymorphism

impl

- Generic
- Method can be inlined
- One pointer
- Single call
- Code duplication

dyn

- Dynamic Dispatch
- Method can't be inlined
- Fat Pointer
- Execution via vtable
- No code duplication

Practice: Pattern Visitor

```
let list: Vec<Box<dyn Location>> =  
vec![  
  Box::new(Tavern),  
  Box::new(Tavern),  
  Box::new(Pit),  
  Box::new(Tavern),  
];
```

```
let human = Human;  
let orc = Orc;
```

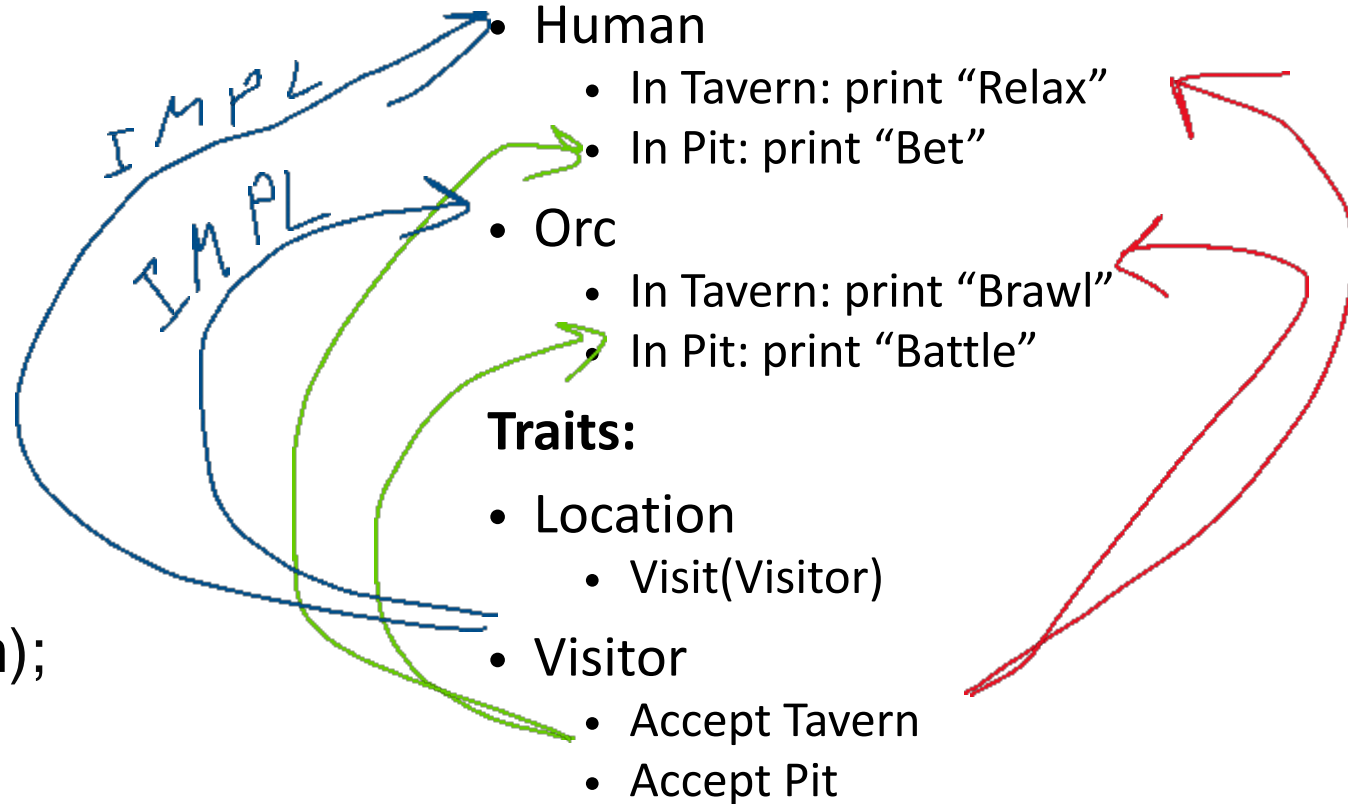
```
for location in &list {  
  location.visit(&human);  
  location.visit(&orc);  
}
```

Entities:

- Human
 - In Tavern: print "Relax"
 - In Pit: print "Bet"
- Orc
 - In Tavern: print "Brawl"
 - In Pit: print "Battle"

Traits:

- Location
 - Visit(Visitor)
- Visitor
 - Accept Tavern
 - Accept Pit



Practice: Pattern Visitor

```
trait Location {  
    fn visit(&self, visitor: &dyn Visitor);  
}  
  
impl Location for Tavern {  
    fn visit(&self, visitor: &dyn Visitor) {  
        visitor.accept_tavern(self);  
    }  
}  
  
impl Location for Pit {  
    fn visit(&self, visitor: &dyn Visitor) {  
        visitor.accept_pit(self);  
    }  
}
```

```
trait Visitor {  
    fn accept_tavern(&self, location: &Tavern);  
    fn accept_pit(&self, location: &Pit);  
}  
  
impl Visitor for Human {  
    fn accept_tavern(&self, _: &Tavern) { println!("Relax") }  
    fn accept_pit(&self, _: &Pit) { println!("Bet") }  
}  
  
impl Visitor for Orc {  
    fn accept_tavern(&self, _: &Tavern) { println!("Brawl") }  
    fn accept_pit(&self, _: &Pit) { println!("Fight") }  
}
```

Packages, Crates, Modules

Package

Crates

Modules

Package vs Crate

- Package – **Cargo** entity
 - Cargo.toml
 - [package]
name = "hello-world"
version = "0.1.0"
edition = "2024"
 - [[bin]]
name = "crate_name"
path = "src/main.rs"
 - [lib]
name = "crate_name"
path = "src/lib.rs"
 - Crate – **rustc** entity
 - src/main.rs
 - src/lib.rs

Crate – compilation unit

- Binary - Multiple

- `[[bin]]`
 `name = "crate_name"`
 `path = "src/main.rs"`
 - `target/debug/crate_name.exe`

- Library - One

- `[lib]`
 `name = "crate_name"`
 `path = "src/lib.rs"`
 - `target/debug/libcrate_name.rlib`

About *.rlib

- Archive
 - .o files - CodeGen Unit
 - LVVM bytecode
 - Metadata
 - Symbol table
 - ...

Crate as rustc entity

- Compilation unit
- Doesn't have name
- May reference other crates
 - foo.rs
 - `pub fn hello(){
 println!("Hello World")
}`
 - bar.rs
 - `fn main(){
 my_foo::hello();
}`

Crate as rustc entity

- foo.rs

- `pub fn hello(){
 println!("Hello World")
}`

`rustc foo.rs --crate-type rlib`

`> libfoo.rlib`

`rustc bar.rs --extern my_foo=./libfoo.rlib`

- bar.rs

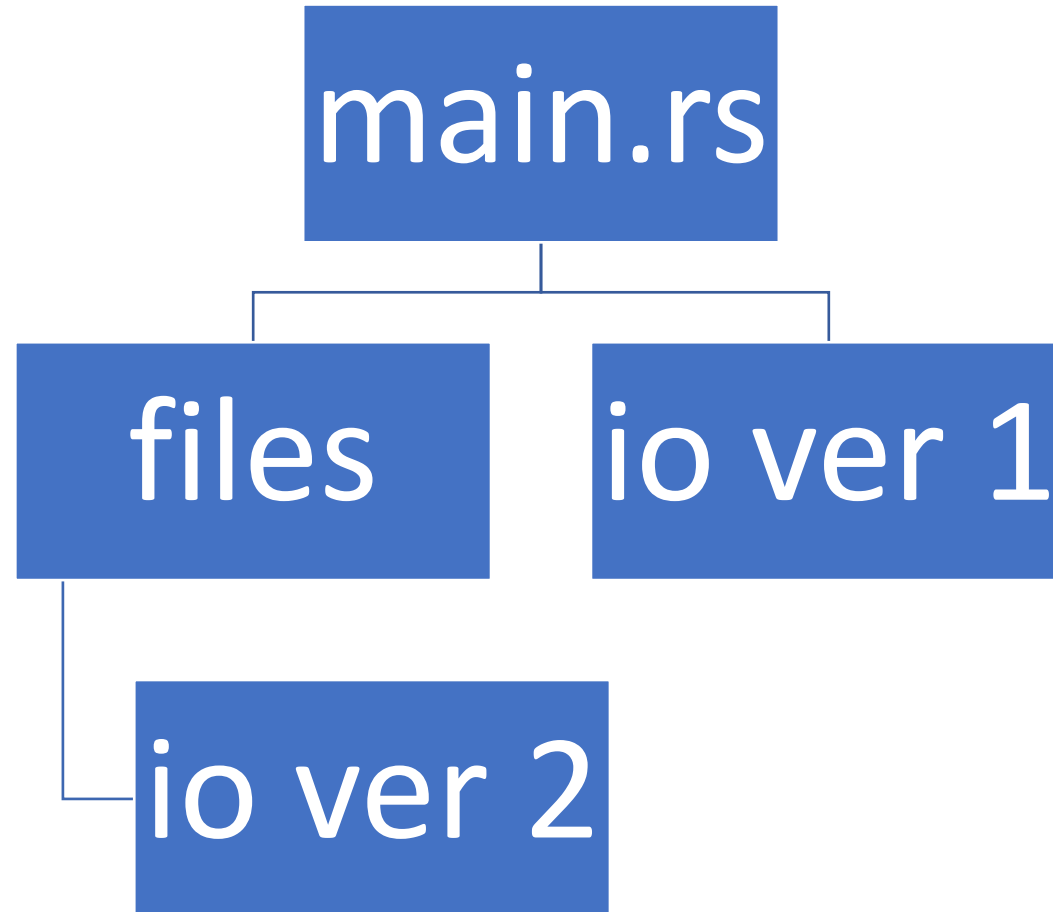
- `fn main(){
 my_foo::hello();
}`

`> bar.exe`

Crate as Cargo entity

- Create == Package with Library Crate
- Cargo.toml
 - [dependencies]
time = "0.1.12"
regex = "0.1.41"
itertools = "0.10.5"

Transitive dependency



SemVer

Given a version number MAJOR.MINOR.PATCH, increment the:

1. MAJOR version when you make incompatible API changes
2. MINOR version when you add functionality in a backward compatible manner
3. PATCH version when you make backward compatible bug fixes

Additional labels for pre-release and build metadata are available as extensions to the MAJOR.MINOR.PATCH format.

SemVer

foo = "1" # >=1.0.0, <2.0.0

foo = "1.2.3" # >=1.2.3, <2.0.0

foo = "^1.2.3" # >=1.2.3, <2.0.0

foo = "~1.2.3" # >=1.2.3, <1.3.0

foo = "=1.2.3" # exactly 1.2.3

Modules

The project

Hello World

- Cargo.toml
- Cargo.lock
- src
 - main.rs

Real

- Cargo.toml
- Cargo.lock
- src
 - main.rs
 - blah-blah
 - dlfsdf.rs
 - sldfsldf.rs
 - abracadabra
 - shwabra
 - ...
 - ...

Crate root

- `main.rs`
- `lib.rs`

Modules in file (main.rs)

```
struct MyStruct0;  
mod level1 {  
    struct MyStruct1;  
    mod level2 {  
        struct MyStruct2;  
    }  
}  
fn main() {  
    let str0 = MyStruct0;  
    let str2 = crate::level1::level2::MyStruct2;  
}
```

error[E0603]: module `level2` is private

Visibility

```
struct MyStruct0;  
mod level1 {  
    struct MyStruct1;  
    fn test(){  
        let parent_str = crate::MyStruct0;  
        let child_str = crate::level2::MyStruct2;  
    }  
  
    mod level2 {  
        struct MyStruct2;  
    }  
}
```

error[E0603]: unit struct `MyStruct2` is private

Visibility

- pub
- pub(crate)
- pub(super)
- pub(self)
- pub(in *path*)

Visibility for everything

```
pub mod module {  
    pub struct MyStruct1{  
        pub value: i32  
    }  
  
    pub fn function(){}  
}
```

pub

```
mod level1 {  
    fn test() {  
        let child_str = crate::level1::level2::MyStruct2;  
    }  
    mod level2 {  
        pub struct MyStruct2;  
    }  
}  
  
fn main() {  
    let str2 = crate::level1::level2::MyStruct2;  
}
```

error[E0603]: module `level2` is private



pub(crate)

- Visible within the current crate
- Most used

pub(super)

```
mod level1 {  
    fn test() {  
        let child_str = crate::level1::level2::MyStruct2;  
    }  
    pub(crate) mod level2 {  
        pub(super) struct MyStruct2;  
    }  
}
```

```
fn main() {  
    let str2 = crate::level1::level2::MyStruct2;  
}
```

error[E0603]: unit struct `MyStruct2` is private

pub(self)

- Private
- Default attribute

pub(in *path*)

```
pub(crate) mod level1 {  
    fn test() {  
        let child_str = crate::level1::level2::level3::level4::MyStruct4; unit struct `MyStruct4` is private  
    }  
  
    pub(crate) mod level2 {  
        fn test() {  
            let child_str = crate::level1::level2::level3::level4::MyStruct4; ✓  
        }  
  
        pub(crate) mod level3 {  
            fn test() {  
                let child_str = crate::level1::level2::level3::level4::MyStruct4; ✓  
            }  
  
            pub(crate) mod level4 {  
                pub(in crate::level1::level2) struct MyStruct4;  
            }  
        }  
    }  
}
```

Absolute and relative path

```
pub(crate) mod level1 {  
    fn test() {  
        let child_str = level2::level3::level4::MyStruct4;  
    }  
  
    pub(crate) mod level2 {  
        fn test() {  
            let child_str = level3::level4::MyStruct4;  
        }  
  
        pub(crate) mod level3 {  
            fn test() {  
                let child_str = level4::MyStruct4;  
            }  
  
            pub(crate) mod level4 {  
                pub(in crate::level1::level2) struct  
MyStruct4;  
            }  
        }  
    }  
}
```

```
pub(crate) mod level1 {  
    fn test() {  
        let child_str = crate::level1::level2::level3::level4::MyStruct4;  
    }  
  
    pub(crate) mod level2 {  
        fn test() {  
            let child_str = crate::level1::level2::level3::level4::MyStruct4;  
        }  
  
        pub(crate) mod level3 {  
            fn test() {  
                let child_str = crate::level1::level2::level3::level4::MyStruct4;  
            }  
  
            pub(crate) mod level4 {  
                pub(in crate::level1::level2) struct MyStruct4;  
            }  
        }  
    }  
}
```

use

```
use crate::level1::level2::level3::level4::MyStruct1;  
use crate::level1::level2::level3::level4::MyStruct2 as  
FooBar;  
use crate::level1::level2::level3::level4::{MyStruct3,  
MyStruct4};  
use crate::level1::level2::level3::level4::*;
```

```
pub(crate) mod level1 {  
    pub(crate) mod level2 {  
        pub(crate) mod level3 {  
            pub(crate) mod level4 {  
                pub(crate) struct  
MyStruct1;  
                pub(crate) struct  
MyStruct2;  
                pub(crate) struct  
MyStruct3;  
                pub(crate) struct  
MyStruct4;  
            }  
        }  
    }  
}
```

Default extern: std

- `use std::prelude::*;`

```
pub use crate::marker::{Send, Sized, Sync, Unpin};
pub use crate::ops::{Drop, Fn, FnMut, FnOnce};
pub use crate::mem::drop;
pub use crate::convert::{AsMut, AsRef, From, Into};
pub use crate::iter::{DoubleEndedIterator, ExactSizeIterator};
pub use crate::iter::{Extend, IntoIterator, Iterator};
pub use crate::option::Option::{self, None, Some};
pub use crate::result::Result::{self, Err, Ok};
pub use core::prelude::v1::{
    assert, cfg, column, compile_error, concat, concat_idents, env, file, format_args,
    format_args_nl, include, include_bytes, include_str, line, log_syntax, module_path, option_env,
    stringify, trace_macros, Clone, Copy, Debug, Default, Eq, Hash, Ord, PartialEq, PartialOrd,
};
pub use core::prelude::v1::concat_bytes;
pub use core::prelude::v1::{RustcDecodable, RustcEncodable};
pub use core::prelude::v1::alloc_error_handler;
pub use core::prelude::v1::{bench, derive, global_allocator, test, test_case};
pub use core::prelude::v1::derive_const;
pub use core::prelude::v1::cfg_accessible;
pub use core::prelude::v1::cfg_eval;
pub use core::prelude::v1::type_ascribe;
pub use crate::borrow::ToOwned;
pub use crate::boxed::Box;
pub use crate::string::{String, ToString};
pub use crate::vec::Vec;
```

Files structure

- Cargo.toml
- Cargo.lock
- src
 - main.rs
 - features.rs
 - magic.rs

- main.rs

```
mod features;  
mod magic;
```

```
fn main() {  
    let feature =  
crate::features::MyFeature;  
}
```

Folders

- Cargo.toml
- Cargo.lock
- src
 - sub_folder
 - sub_features.rs
 - main.rs
 - features.rs
 - magic.rs

Folders

- Cargo.toml
- Cargo.lock
- src
 - sub_folder.rs
 - sub_folder
 - sub_features.rs
 - main.rs
 - features.rs
 - magic.rs

- sub_folder.rs

```
mod sub_feature;
```

- main.rs

```
mod sub_folder;
```

```
use sub_folder::sub_feature::*;
```

```
fn main() {
```

```
    let feature = MySubFeature;
```

```
}
```

Folders

- Cargo.toml
- Cargo.lock
- src
 - sub_folder.rs
 - sub_folder
 - sub_feature.rs
 - main.rs
 - features.rs
 - magic.rs

- sub_folder.rs

```
pub(crate) mod sub_feature;
```

- main.rs

```
mod sub_folder;
```

```
use sub_folder::sub_feature::*;
```

```
fn main() {
```

```
    let feature = MySubFeature;
```

```
}
```

Homework

- Rustlings:
 - traits
- Courseware:
 - New Feature: print to file -f “filename”
 - `cargo run -- c:/Projects --find main.rs --sort -f output.txt`
 - Create new file output.txt
 - Print all to this file
 - DON'T ABUSE STDOUT!!!!
 - Use pattern “Strategy”: <https://refactoring.guru/design-patterns/strategy>
 - Refactor code to modules
 - Add visibility modifiers